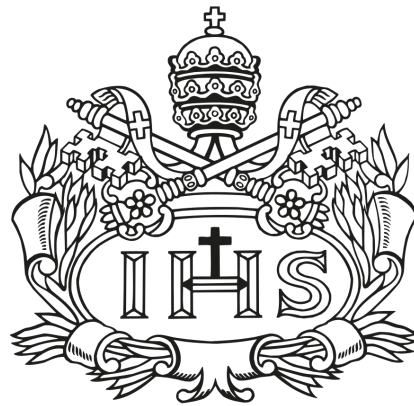


Pontificia Universidad Javeriana



Pontificia Universidad
JAVERIANA
Colombia

Taller Diseño

Grupo: 2

Integrantes:

Jonathan Martínez Gómez
jonathanmartinez@javeriana.edu.co

Juan David Acuña Lesmes
jdavidacuna@javeriana.edu.co

Samuel Bonilla Bravo
bonillasamuel@javeriana.edu.co

Juan Guillermo Gómez
gomez.juang@javeriana.edu.co

Docente:

Ing. Luis Gabriel Moreno Sandoval, PhD
Fundamentos de Ingeniería de Software

23 de Septiembre de 2025

Índice general

1. INTRODUCCIÓN	1
1.1. Objetivo General	1
1.2. Objetivos Específicos	1
2. Diseño del Sistema	3
2.1. Diagrama de Análisis (ECB)	3
2.1.1. Descripción	3
2.1.2. Representación Gráfica	4
2.2. Diagrama de Clases	4
2.2.1. Descripción	4
2.2.2. Representación Gráfica	5
2.3. Diagrama de Componentes	5
2.3.1. Descripción	5
2.3.2. Representación Gráfica	6
2.4. Diagrama de Despliegue	7
2.4.1. Descripción	7
2.4.2. Representación Gráfica	7
2.5. GitHub: Descripción y Evidencias de Issues y Commits Asociados	8
2.5.1. Diagrama de Análisis (ECB)	8
2.5.2. Diagrama de Clases	8
2.5.3. Diagrama de Componentes	9
2.5.4. Diagrama de Despliegue	10
3. CONCLUSIONES	11
3.1. Conclusiones	11

Capítulo 1

INTRODUCCIÓN

El presente documento tiene como propósito describir el proceso de diseño del sistema **FIS (Sistema de Gestión de Información en Urgencias)**, cuyo objetivo principal es apoyar la atención hospitalaria en el área de admisiones y urgencias. Este sistema busca garantizar que, desde el primer contacto del paciente con el centro de salud, se pueda realizar un registro ágil, completo y estructurado de su información básica, sus signos vitales y su clasificación por nivel de triage.

El desarrollo de FIS se enmarca en el taller de diseño en software, donde se aplican metodologías de modelado basadas en UML. A través de diagramas de análisis, clases, componentes y despliegue, se pretende obtener una representación clara de la arquitectura del sistema, sus interacciones internas y externas, y la infraestructura tecnológica necesaria para su implementación. De esta forma, se busca asegurar que el sistema pueda evolucionar de manera organizada y que los distintos actores involucrados cuenten con una herramienta confiable para la toma de decisiones clínicas y administrativas.

1.1. Objetivo General

Definir y documentar los requisitos funcionales y no funcionales del sistema **FIS**, mediante la redacción de historias de usuario y la elaboración de casos de uso, con el fin de estructurar de manera clara las necesidades de los distintos stakeholders del área de urgencias y asegurar una base sólida para el desarrollo del sistema.

1.2. Objetivos Específicos

1. Identificar a los stakeholders del sistema (personal de admisión, personal de enfermería en urgencias, seguridad del sistema, usabilidad y experiencia de usuario) y derivar sus necesidades en historias de usuario siguiendo el formato establecido.
2. Redactar historias de usuario representativas para cada stakeholder, incluyendo el registro de datos mínimos de identificación, la documentación de signos vitales y la asignación de niveles de triage, junto con historias de requisitos no funcionales y una adicional proveniente de consulta externa.
3. Diseñar casos de uso detallados que incluyan nombre, actores, descripción, flujo principal y flujos alternativos, con el fin de precisar los escenarios de interacción del sistema.

4. Elaborar el conjunto de diagramas de diseño —ECB, clases, componentes y despliegue— que permitan representar de manera estructurada la arquitectura de FIS, sus módulos principales y la infraestructura tecnológica requerida.

Capítulo 2

Diseño del Sistema

2.1. Diagrama de Análisis (ECB)

2.1.1. Descripción

El diagrama ECB del sistema FIS muestra cómo el Personal Médico interactúa con los formularios de admisión, signos vitales y triage (Boundaries). Cada formulario se comunica con un Control correspondiente (RegistrarPacienteCtrl, RegistrarSignosCtrl y AsignarTriageCtrl), encargado de validar la información y coordinar la creación o consulta de las entidades del dominio.

Las **Entidades** principales representadas son:

- **Paciente** y su **Ingreso**, que permiten identificar y asociar los registros.
- **SignosVitales**, documentados al ingreso por el personal de enfermería.
- **Triage**, que clasifica al paciente según la gravedad de su estado.
- **Usuario**, que valida los permisos y roles de quien realiza las acciones.

De esta forma, el modelo refleja el flujo central de la atención en urgencias: registro de datos básicos, documentación de signos vitales y asignación de un nivel de triage, garantizando trazabilidad y validación por rol.

2.1.2. Representación Gráfica

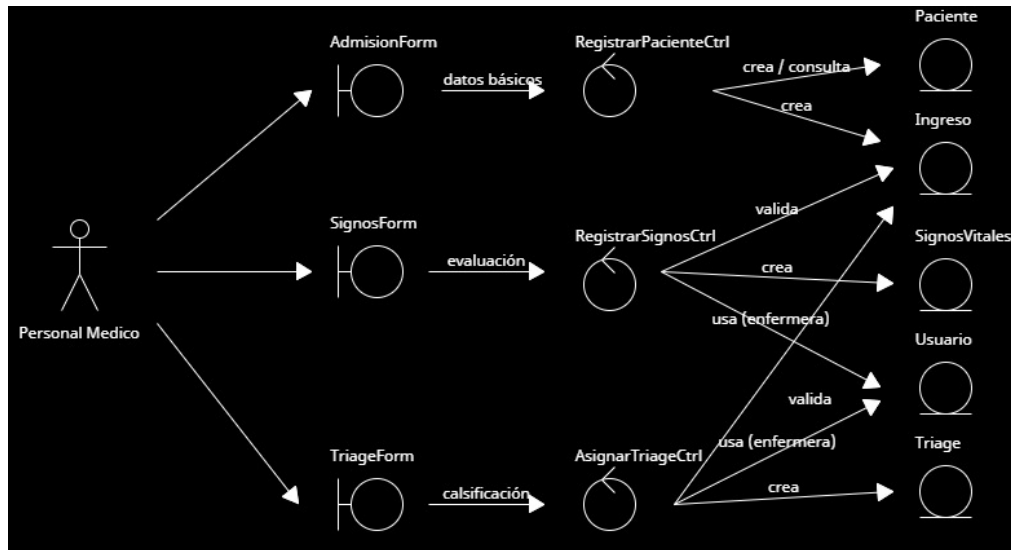


Figure 2.1: Diagrama de Análisis basado en ECB

2.2. Diagrama de Clases

2.2.1. Descripción

El modelo de clases está diseñado siguiendo el patrón BCE (Boundary – Control – Entity), lo que garantiza una separación clara entre la interfaz de usuario, la lógica de negocio y los datos persistentes. Este diseño mejora el mantenimiento del sistema, facilita las pruebas y asegura un código más ordenado y modular.

■ Principales clases identificadas:

- **Boundaries:** SignosForm, TriageForm, AdmisionForm.
- **Controls:** RegistrarSignosCtrl, AsignarTriageCtrl, RegistrarPacienteCtrl.
- **Entities:** SignosVitales, Triage, Paciente, Ingreso, Usuario.

■ Atributos y métodos principales:

- **SignosVitales:** *temperatura, frecuenciaCardiaca*; *+esCritico(): boolean.*
- **Triage:** *nivel*; *+esVigente(): boolean.*
- **Paciente:** *nombre, documento*; *+actualizarDatos(): void.*
- **Ingreso:** *estado*; *+cerrarIngreso(): void.*
- **Usuario:** *nombre, rol*; *+autenticar(): boolean.*
- **Controladores:** métodos como *registrarSignos()*, *asignarTriage()*, *registrarPaciente()* coordinan las operaciones sobre las entidades.

■ Relaciones:

- **Composición:** un Paciente puede tener uno o varios Ingreso; un Ingreso agrupa varios SignosVitales y Triage.

- **Agregación:** los controladores interactúan con las entidades para leer y modificar información, pero no las poseen de forma exclusiva.
- **Herencia:** en este modelo no se utiliza herencia, ya que cada clase tiene un rol bien definido.

2.2.2. Representación Gráfica

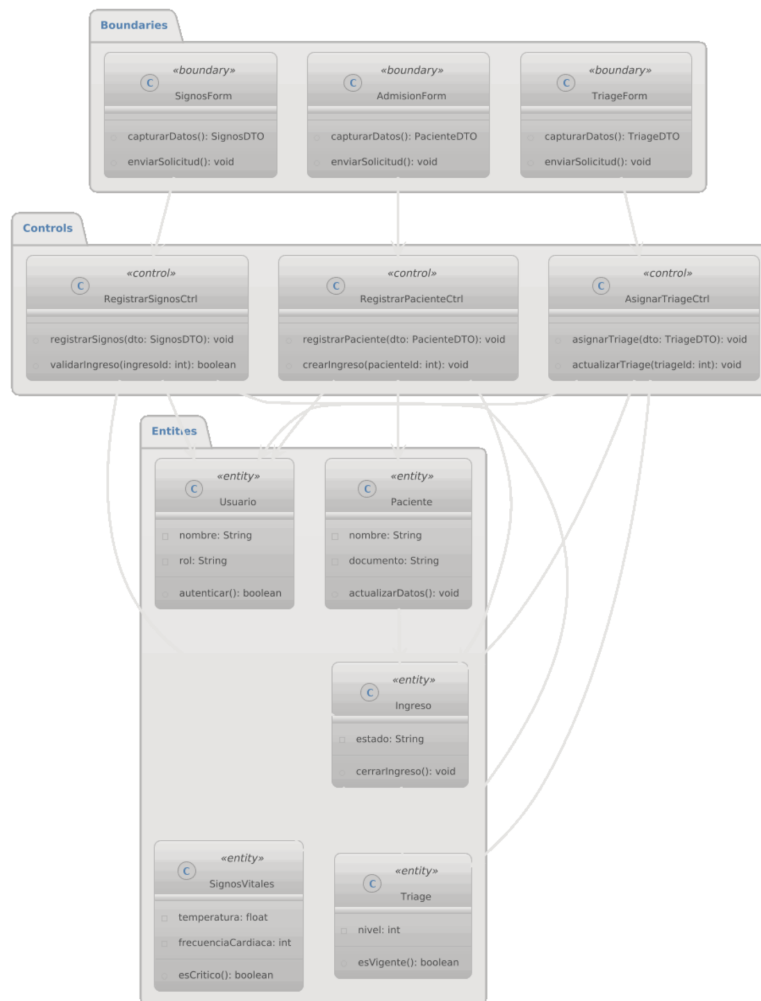


Figure 2.2: Diagrama de Clases del sistema

2.3. Diagrama de Componentes

2.3.1. Descripción

El sistema fue modelado siguiendo el patrón Boundary-Control-Entity, lo que permitió asegurar una alta cohesión y bajo acoplamiento entre los módulos.

- **Los componentes principales son:**
Interfaces de usuario (SignosUI, TriageUI, AdmisionUI): Son las pantallas que sirven para capturar datos y enviar solicitudes.

Lógica de negocio(Módulo Signos, Módulo Triage, Módulo Pacientes): Funciona como un mediador entre la UI y el dominio, aplicando las reglas del negocio de cada proceso hospitalario. El módulo de signos se encarga de la gestión de registro y validación de signos vitales. El módulo de Triage se encarga de la gestión de asignación y actualización de niveles de triage. Por último, el módulo de Pacientes se encarga de la gestión de registro y admisión de pacientes.

Dominio(Gestión Clínica, Gestión de Pacientes, Gestión de Ingresos, Gestión de Usuarios): Representan la información más importante del sistema, junto a sus relaciones. La gestión de pacientes es equivalente a la entidad paciente, el de ingresos a entidad ingresos, el de clínica al de Signos Vitales y Triage y por último el de Usuarios a la entidad Usuario. Las interfaces de usuario exponen métodos para capturar datos y enviarlos al sistema.

■ **Interfaces utilizadas y expuestas:**

Los controladores exponen operaciones de negocio como registrarPaciente(), asignarTriage() o registrarSignos(). El módulo de dominio expone interfaces para acceder a las entidades, permitiendo ejecutar métodos como esCritico(), cerrarIngreso() o autenticar(). De esta manera, cada capa interactúa únicamente mediante interfaces definidas, asegurando encapsulamiento y modularidad.

■ **Conexión con servicios externos (APIs, BD, etc.):**

No tiene servicios externos.

2.3.2. Representación Gráfica

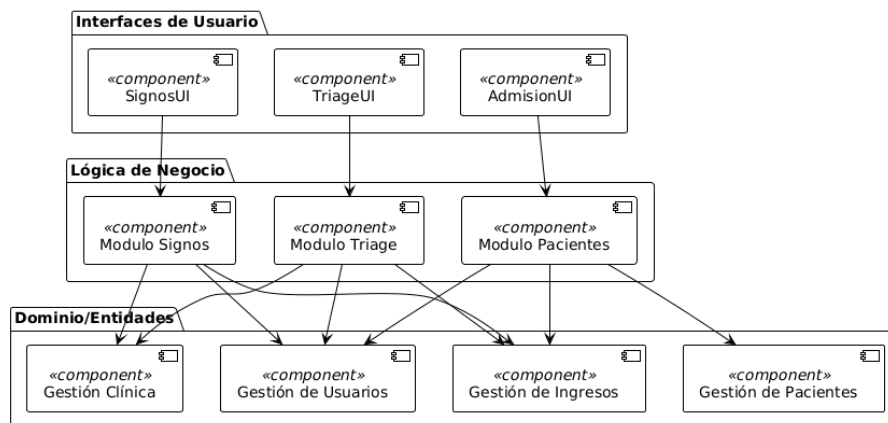


Figure 2.3: Diagrama de Componentes del sistema

2.4. Diagrama de Despliegue

2.4.1. Descripción

El diagrama de despliegue muestra, en tiempo de ejecución, la distribución de los artefactos de la aplicación sobre tres nodos principales: el Cliente donde el Personal Médico interactúa con los formularios AdmissionForm, SignosForm y TriageForm; el backend (aplicación MVC) que expone los controllers RegistrarPacienteCtrl, RegistrarSignosCtrl y AsignarTriageCtrl y contiene la lógica de negocio; y la base de datos que se planea trabajar con PostgreSQL que persiste las entidades Paciente, Ingreso, SignosVitales, Usuario y Triage. En ejecución el flujo típico es: el Personal Médico completa un formulario en el Cliente y éste envía una petición al backend; el controller correspondiente valida y procesa la petición y, dentro de una transacción atómica, ejecuta sentencias SQL contra PostgreSQL para crear o consultar las tablas indicadas; finalmente el backend devuelve la respuesta al Cliente. Las dependencias representadas indican qué controllers crean o usan qué entidades (por ejemplo, RegistrarPacienteCtrl crea/consulta Paciente e Ingreso; RegistrarSignosCtrl crea SignosVitales y asocia Usuario; AsignarTriageCtrl crea Triage y referencia Usuario), lo que ayuda a identificar responsabilidades y puntos críticos para transacciones, permisos y auditoría.

2.4.2. Representación Gráfica

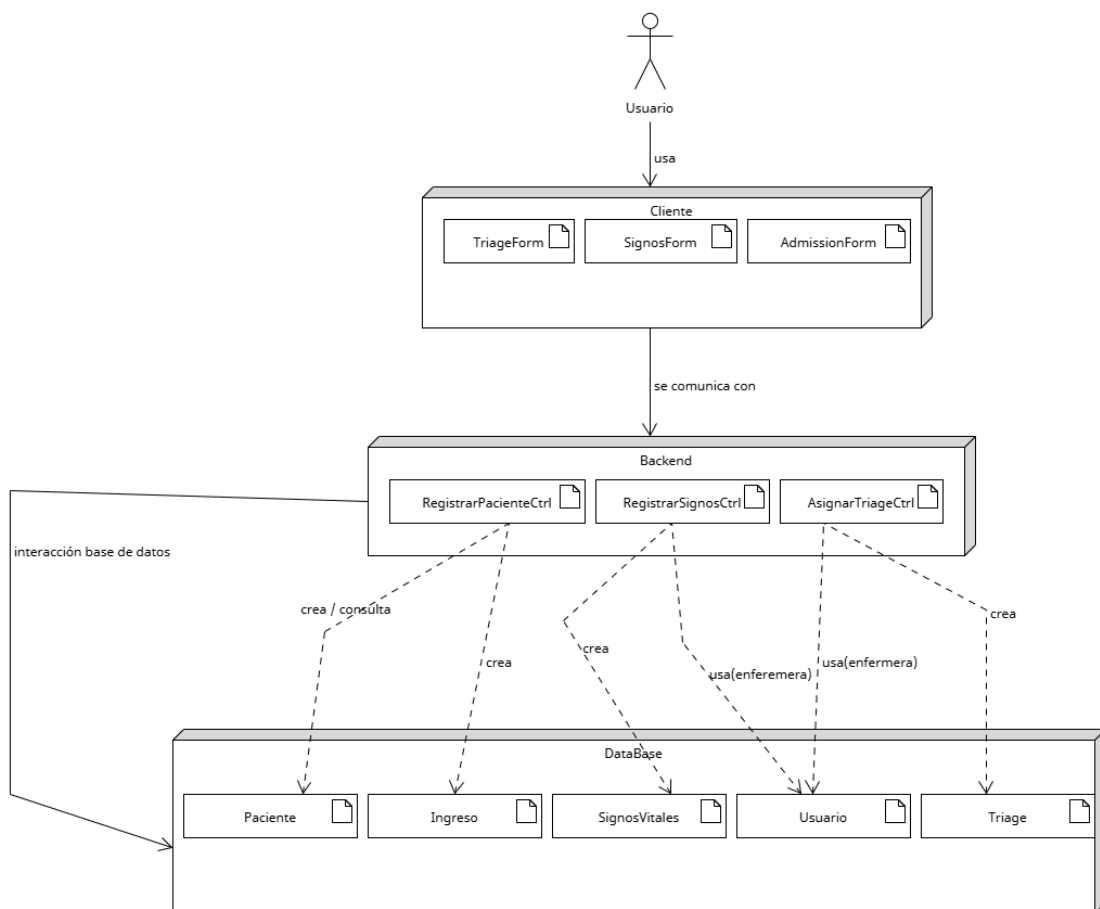


Figure 2.4: Diagrama de Despliegue del sistema

2.5. GitHub: Descripción y Evidencias de Issues y Commits Asociados

En esta sección se presentan los issues creados en el repositorio de GitHub, junto con sus evidencias y commits asociados. Cada issue corresponde a uno de los diagramas solicitados en el taller de diseño del sistema FIS.

2.5.1. Diagrama de Análisis (ECB)

Descripción: Este issue corresponde al diseño del diagrama de análisis bajo el patrón ECB, donde se modelan los límites (formularios), controles (coordinadores de casos de uso) y entidades (Paciente, Ingreso, SignosVitales, Triage, Usuario).

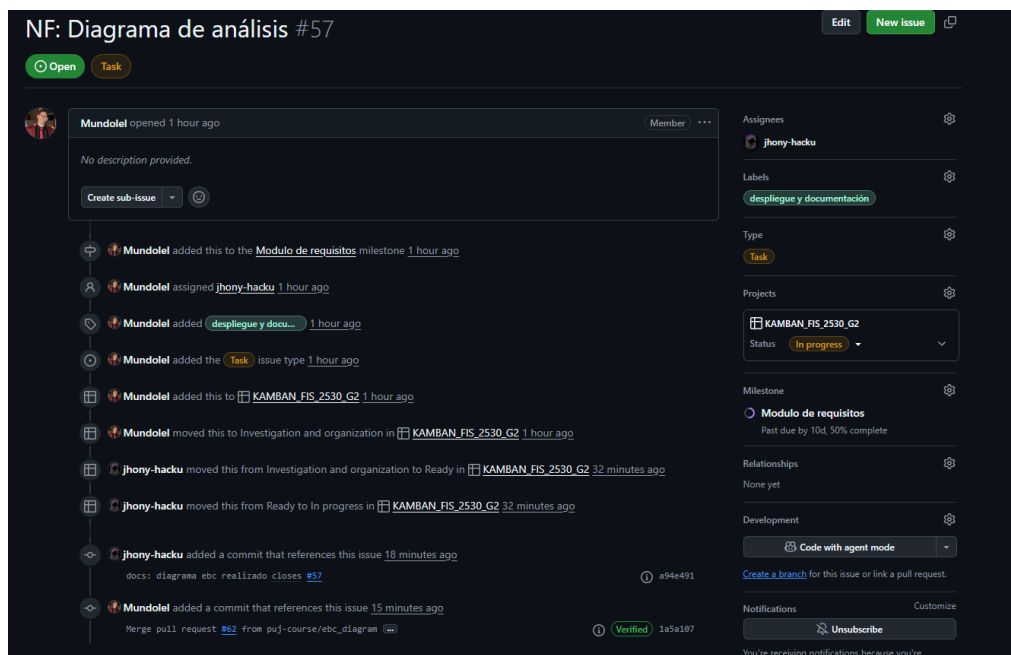


Figure 2.5: Evidencia del Issue y Commit asociado al Diagrama de Análisis (ECB)

2.5.2. Diagrama de Clases

Descripción: Este issue corresponde al diseño del modelo de clases del sistema FIS, detallando atributos, métodos y relaciones (herencia, composición, agregación) entre las entidades principales.

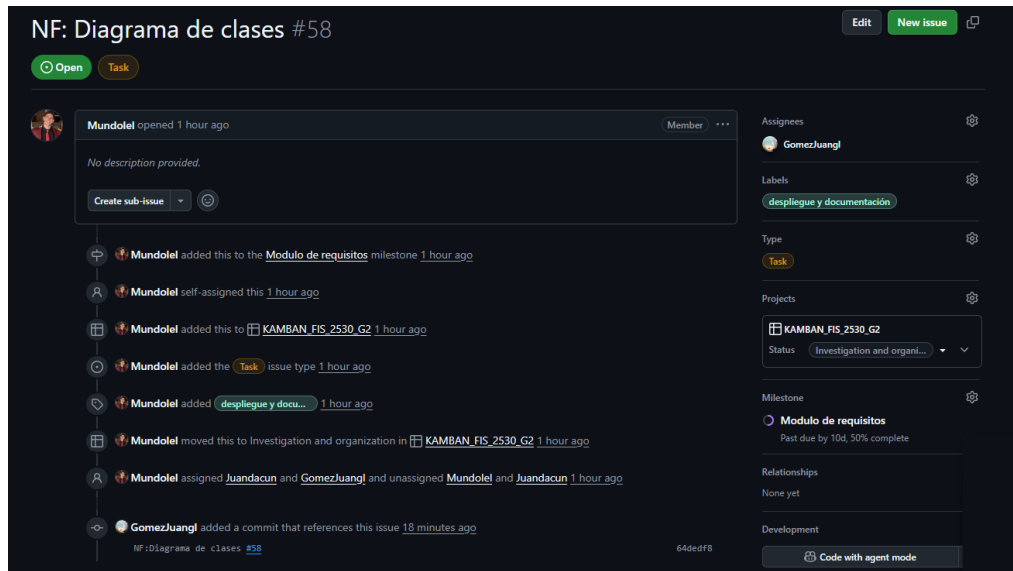


Figure 2.6: Evidencia del Issue y Commit asociado al Diagrama de Clases

2.5.3. Diagrama de Componentes

Descripción: Este issue corresponde al diseño del diagrama de componentes, donde se representan los módulos internos del sistema, las interfaces utilizadas y expuestas, así como la interacción con servicios externos y bases de datos.



Figure 2.7: Evidencia del Issue y Commit asociado al Diagrama de Componentes

2.5.4. Diagrama de Despliegue

Descripción: Este issue corresponde al diseño del diagrama de despliegue, donde se especifica la infraestructura necesaria (servidores, contenedores, nube), junto con la comunicación entre frontend, backend y servicios externos.

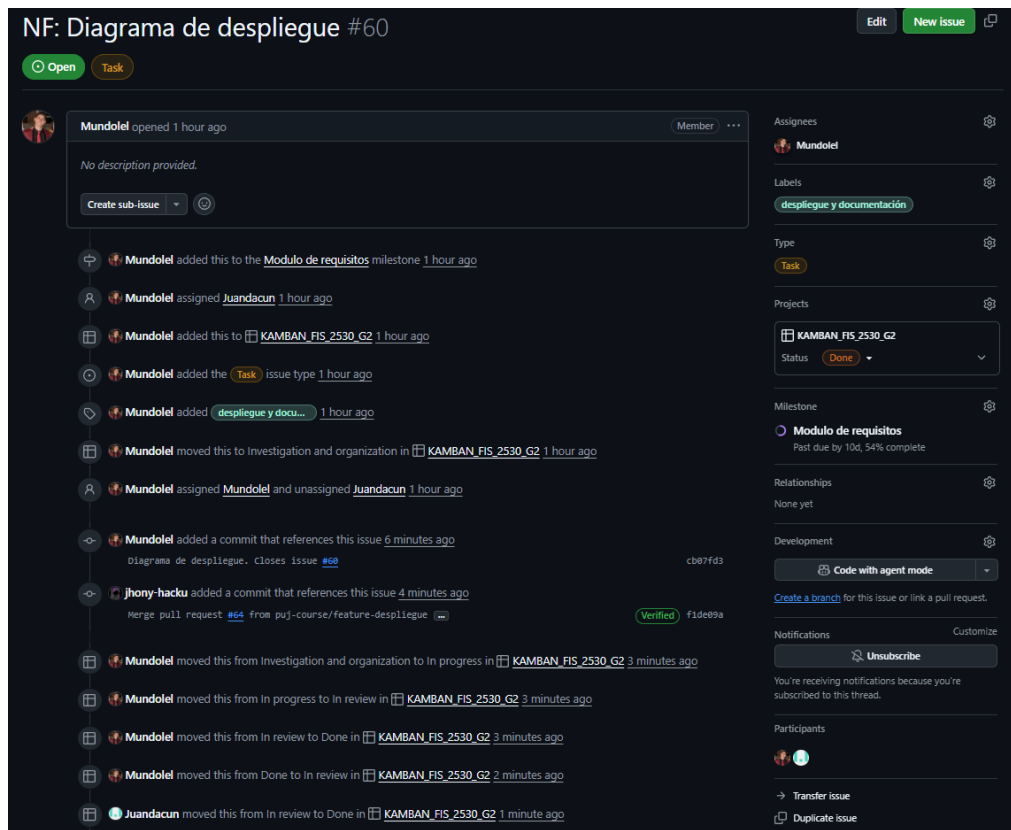


Figure 2.8: Evidencia del Issue y Commit asociado al Diagrama de Despliegue

Capítulo 3

CONCLUSIONES

3.1. Conclusiones

El desarrollo del sistema FIS (Sistema de Gestión de Información en Urgencias) permitió aplicar de manera práctica los conceptos de modelado y diseño de software vistos en el curso. A través de la elaboración de diagramas ECB, de clases, de componentes y de despliegue, se logró representar de forma clara la estructura del sistema, sus interacciones y la infraestructura tecnológica necesaria para su implementación.

El uso de historias de usuario como punto de partida facilitó la identificación de los distintos stakeholders (personal de admisión, enfermería, seguridad del sistema, usabilidad y experiencia de usuario) y permitió traducir sus necesidades en casos de uso representativos. Este enfoque garantizó trazabilidad entre los requisitos y los modelos generados, fortaleciendo la coherencia del diseño.

En conclusión, el taller permitió comprender la importancia de un diseño estructurado y documentado antes de la implementación, resaltando cómo los modelos UML apoyan la comunicación entre los miembros del equipo, la validación con los stakeholders y la construcción de una base sólida para el desarrollo futuro del sistema.

Bibliografía